

SkyvoltRobot MVP

Phase 1 보고서 — 위치추정 견고화 (Hardening Localization)

작성일: 2026 - 06 - 15

1. Phase 1 개요

Phase 1의 목표는 '위치추정(localization)을 견고하게' 만드는 것이다. 핵심 IP인 트랙 - 호장(arclength) 칼만필터와 도킹 속도정책 FSM을 더 정확하고, 더 깨지지 않고, 재현 가능하게 다듬었다. 로드맵의 4개 작업 중 소프트웨어 작업 3건을 완료했고, 실물 RFID 리더 측정이 필요한 #1만 하드웨어 의존으로 보류했다.

작업	성격	상태
#2 IMU + 휠 엔코더 융합 (곡선 추정)	순수 SW	완료
#3 속성 기반 fault - injection 테스트	순수 SW	완료
#4 Replay 도구 (bag → 오프라인 → 동일 지표)	순수 SW	완료
#1 실측 RFID 노이즈 모델	HW 필요	보류

Phase 1에서 속성·IMU·Replay 테스트 약 22개를 추가했고, 저장소 전체 테스트는 전부 통과한다(현재 65개).

2. #2 — IMU + 휠 엔코더 융합

칼만필터(상태 $[s, ds] = \text{호장, 호장속도}$)를 확장하되 기존 API와 완전한 하위 호환을 유지했다. 두 갈래로 IMU를 융합한다:

- 가속도계 → $\text{predict}()$ 의 제어 입력: 측정 가속도가 평균을 구동(등가속도 모델)해, 속도가 바뀌는 구간(곡선 진입·접근 감속) 추정을 개선한다.
- 자이로 → 곡선 속도 측정: 반경 R 곡선에서 $v = \omega \cdot R$ 로 엔코더와 독립적인 속도를 융합해 휠 슬립에 강인하다. 직선($R=\infty$)에선 자동 무시.

$\text{predict}(dt)$ 에 인자를 주지 않으면 수학적으로 이전과 완전히 동일하므로($Q = B \cdot B^T \cdot \sigma^2$), 도킹 평가 게이트는 그대로 PASS를 유지한다. ROS 노드는 `/imu(sensor_msgs/Imu)`를 구독하며, IMU가 끊기면 등속도 모델로 자동 폴백한다.

효과 입증 (단위 테스트)

동일한 노이즈 엔코더 스트림에 대해, 가속 구간에서 IMU 융합 필터가 등속도 필터보다 속도 추정 오차가 작음을 테스트로 검증했다 (`test_imu_accel_beats_constant_velocity_on_a_ramp`). 자이로가 곡선에서 속도를 고정하고 직선에선 무동작함도 검증.

3. #3 — 속성 기반 fault-injection 테스트

손으로 고른 몇 개 입력 대신, Hypothesis가 무작위·극단 입력 시퀀스를 자동 생성해 칼만필터와 FSM이 '어떤 유한 입력에도' 지켜야 할 불변식을 검증한다.

칼만필터 불변식

- 공분산은 항상 대칭 + 양의 준정부호(PSD)를 유지한다.
- 유한 입력에 대해 추정값·공분산이 NaN/Inf가 되지 않는다.
- 측정 보정은 관측 차원의 분산을 절대 키우지 않는다(KF 근본 성질).
- 정밀 RFID 고정은 추정값을 측정값 쪽으로만 끌어당긴다.

- `predict(dt ≤ 0)`은 상태를 바꾸지 않는다(무동작).

FSM 불변식

- 명령 속도는 항상 $[0, 1.5]$ m/s 범위이며, 비상정지 외에는 저크 제한을 지킨다.
- 도킹 단계는 전진만 한다(CRUISE→A1→A2→DOCKED), 역행 없음.
- 터미널 상태(DOCKED/FAULT)는 고착되고 0으로 단조 감속한다.
- photoeye 발화 시 즉시 DOCKED로 latch한다(비상정지).

성과: 속성 테스트가 처음 작성한 불변식 하나가 과하게 강했음을 즉시 잡아냈다(정상적인 비상정지 감속을 위반으로 오판). FSM 코드 자체는 정상이었고 명세를 바로잡았다 — 이것이 속성 기반 테스트의 가치다.

4. #4 — Replay 도구 (bag → 오프라인 localizer → 동일 지표)

기록된 센서 스트림을 ROS/Gazebo 없이 오프라인 localizer로 재생해 동일한 위치추정 지표를 재현한다. ROS 독립 JSONL trace 포맷(특별 odom·imu·gyro·rfid·photoeye + 지상진실)을 정의하고, rosbag2를 읽어 trace로 변환하는 도구(ROS 있을 때)와 순수 파이썬 재생·지표 산출(CI 가능)을 분리했다.

핵심 보증 — 비트 단위 재현

localizer는 결정론적이므로, 기록된 입력을 오프라인 재생하면 라이브 추정과 비트 단위로 동일해야 한다. 504틱 트레이스에서 불일치 0을 확인했다(테스트로 보장).

```
$ python -m skyvolt_eval.replay --record t.jsonl --imu
wrote 504 ticks -> t.jsonl
$ python -m skyvolt_eval.replay --trace t.jsonl
{"loc_err_mm": { "p50": ..., "p99": ... },
"final_loc_err_mm": 1.34          # photoeye anchors tight at dock
```

5. #1 — 실측 RFID 노이즈 모델 (보류)

합성 RFID 노이즈 모델을 실측 분포로 교체하는 작업은 벤치탑 RFID 리더(IQT1 - FP - R4 - V1) 측정이 필요하다. 측정 데이터를 불러와 적용하는 코드 구조는 미리 만들 수 있으나, 실제 분포는 하드웨어 확보 후 채운다. 현재는 보류 상태이며 Phase 1의 유일한 미완 항목이다.

6. Phase 1 추가 / 변경 파일

파일	구분	내용
track_localizer.py	수정	IMU 가속도 제어입력 + 자이로 곡선 속도 융합
localizer_node.py / package.xml	수정	/imu 구독(sensor_msgs)
skyvolt_eval/replay.py	신규	Replay 도구(trace·재생·지표·bag 변환)
test_track_localizer_properties.py	신규	KF 속성 테스트
test_speed_policy_properties.py	신규	FSM 속성 테스트
test_replay.py	신규	Replay 비트단위 재현 테스트
test_track_localizer.py	수정	IMU·자이로 단위 테스트 추가

7. 실행 방법

```
cd skyvolt_mvp
./scripts/run_tests.sh          # all tests (65)
./scripts/run_docking_eval.sh   # KF backward-compat PASS
python -m skyvolt_eval.replay --record t.jsonl # record a trace
```

```
python -m skyvolt_eval.replay --trace t.jsonl # replay -> metrics
```

8. 결론

- Phase 1의 소프트웨어 작업 3건을 완료 — IMU 융합으로 추정 정확도를 높이고, 속성 테스트로 견고성을 보장하고, Replay로 재현·디버깅 기반을 마련했다.
- 기존 도킹 평가 게이트는 하위 호환으로 그대로 PASS를 유지한다.
- 잔여 항목은 하드웨어 의존인 #1(실측 RFID)뿐이다.